Proyecto 02 Introducción a la Inteligencia Artificial

Viviana Gomez Leon, Paula Gabriela Losada Campo, Carol Alexa Robayo Fuentes, Luis Eduardo Diaz Amado
Departamento de Ingeniería de Sistemas
Pontificia Universidad Javeriana
Bogotá, Colombia

***Resumen*—El proyecto consiste en desarrollar un programa capaz de resolver un sudoku utilizando el enfoque de problemas de satisfacción de restricciones (CSP). Se implementan diferentes estrategias de solución, incluyendo fuerza bruta, backtracking y backtracking con comprobación hacia adelante, con el fin de encontrar una solución válida que cumpla las restricciones del juego. Además, se realiza un análisis comparativo del rendimiento de cada método para determinar cuál es más eficiente.**

I. Introducción

El presente proyecto aborda la resolución del juego Sudoku mediante técnicas de Inteligencia Artificial, específicamente a través del enfoque de problemas de satisfacción de restricciones (CSP). Este tipo de problemas se caracteriza por la necesidad de encontrar una asignación de valores a un conjunto de variables que cumpla con un conjunto de restricciones previamente definidas. En el caso del Sudoku, dichas restricciones implican que no se repitan números en filas, columnas ni subcuadrículas.

Para la solución del problema, se implementan diferentes estrategias algorítmicas, entre ellas la fuerza bruta, el algoritmo de backtracking y una versión optimizada con comprobación hacia adelante. Estas técnicas permiten explorar el espacio de búsqueda de manera sistemática, mejorando la eficiencia en la obtención de soluciones válidas. Además, el proyecto incluye un análisis comparativo del desempeño de cada método, con el fin de identificar sus ventajas y limitaciones en términos de complejidad y tiempo de ejecución.

Este trabajo permite comprender la importancia de los CSP en la Inteligencia Artificial y su aplicabilidad en problemas reales, donde la optimización y el cumplimiento de restricciones son fundamentales.

II. Objetivo

Desarrollar un programa en Python capaz de resolver un sudoku mediante el uso de técnicas de satisfacción de restricciones (CSP), implementando diferentes algoritmos como fuerza bruta, backtracking y backtracking con comprobación hacia adelante, con el fin de obtener soluciones válidas y analizar el rendimiento de cada método.

III. Marco Teórico

La Inteligencia Artificial (IA) es una disciplina que se encarga del desarrollo de sistemas capaces de simular procesos de razonamiento y resolución de problemas. Dentro de este campo, los problemas de satisfacción de restricciones (CSP, Constraint Satisfaction Problems) constituyen una técnica fundamental para modelar situaciones en las que se deben cumplir múltiples condiciones de manera simultánea [1].

Un CSP se define mediante tres componentes principales: un conjunto de variables, un dominio de valores posibles para cada variable y un conjunto de restricciones que limitan las combinaciones de valores que dichas variables pueden tomar. La solución de un CSP consiste en encontrar una asignación completa de valores que satisfaga todas las restricciones establecidas. Este tipo de problemas es ampliamente utilizado en áreas como la planificación, la programación de horarios, la asignación de recursos y la resolución de juegos lógicos [2].

El Sudoku es un ejemplo clásico de problema CSP. En este contexto, cada celda del tablero representa una variable, mientras que su dominio está compuesto por los números del 1 al 9. Las restricciones del problema establecen que no se deben repetir valores en una misma fila, columna o subcuadrícula. Esta estructura convierte al Sudoku en un caso ideal para aplicar técnicas de búsqueda y razonamiento dentro de la Inteligencia Artificial [1].

Uno de los enfoques más básicos para resolver este tipo de problemas es la fuerza bruta, que consiste en evaluar todas las posibles combinaciones de valores hasta encontrar una solución válida. Aunque este método garantiza encontrar una solución si existe, presenta una complejidad computacional elevada, debido al crecimiento exponencial del número de combinaciones posibles [3].

Para mejorar la eficiencia en la resolución de CSP, se emplea el algoritmo de backtracking, el cual realiza una búsqueda sistemática en el espacio de soluciones. Este algoritmo construye soluciones de manera incremental y, en cada paso, verifica si la asignación parcial cumple con las restricciones. En caso contrario, se descarta dicha asignación y se retrocede para explorar otras alternativas, reduciendo así el número de combinaciones a evaluar [1].

Una extensión del backtracking es la técnica de comprobación hacia adelante (forward checking), la cual consiste en analizar las variables no asignadas y eliminar de sus dominios aquellos valores que podrían generar conflictos en el futuro. Este proceso permite detectar inconsistencias de manera anticipada, evitando la exploración de caminos que no conducen a soluciones válidas [2].

Adicionalmente, en los problemas CSP es común utilizar heurísticas que optimizan la búsqueda, como la selección de la variable más restringida o la elección del valor menos restrictivo. Estas estrategias permiten reducir el tamaño del

espacio de búsqueda y mejorar el desempeño de los algoritmos [1].

Un aspecto fundamental en el análisis de algoritmos de satisfacción de restricciones es el Overhead Computacional, el cual se refiere al costo adicional asociado a la gestión de estructuras auxiliares y a operaciones que no forman parte directa de la construcción de la solución, como la verificación de restricciones, el mantenimiento de dominios y la propagación de información entre variables.

En problemas CSP, este overhead se hace evidente en técnicas como el forward checking, donde es necesario actualizar dinámicamente los dominios de las variables y propagar restricciones hacia sus vecinos después de cada asignación. Estas operaciones incrementan el costo computacional por paso en comparación con enfoques más simples como la fuerza bruta o el backtracking básico.

Sin embargo, este aumento en el costo por operación introduce un Trade-off entre costo y eficiencia. Por un lado, los algoritmos más avanzados requieren mayor procesamiento en cada iteración debido al manejo de información adicional; por otro lado, logran reducir significativamente el espacio de búsqueda al evitar la exploración de ramas inválidas desde etapas tempranas.

Las técnicas de propagación de restricciones incrementan el trabajo realizado en cada paso del algoritmo, pero permiten detectar inconsistencias de manera anticipada, reduciendo el número total de nodos explorados en el árbol de búsqueda [1], [2]. En este sentido, el rendimiento de un algoritmo no depende únicamente del tiempo de ejecución por operación, sino también de la cantidad total de operaciones necesarias para alcanzar una solución.

Por ello, algoritmos como el forward checking, a pesar de presentar un mayor overhead computacional, suelen ser más eficientes en términos globales, ya que disminuyen drásticamente el número de nodos explorados. Este equilibrio entre el costo computacional por paso y la reducción del espacio de búsqueda constituye un principio clave en el diseño de algoritmos para problemas CSP.

IV. Procedimiento

En esta sección se describe la metodología empleada para la resolución del problema del Sudoku utilizando técnicas de satisfacción de restricciones (CSP). Se detallan los criterios utilizados para la representación del tablero, la definición de variables, dominios y restricciones, así como las estrategias implementadas para la búsqueda de soluciones.

IV-A. Representación del Problema

El problema del Sudoku se modela como un problema de satisfacción de restricciones (CSP), donde el tablero se representa mediante una **matriz bidimensional de tamaño 9×9**, implementada en Python como una lista de listas. Cada posición de la matriz corresponde a una celda del Sudoku, donde los valores del 1 al 9 representan números asignados, y el valor 0 indica una celda vacía que debe ser completada.

IV-A1. Matriz 9×9: El tablero se maneja como una estructura de datos de la siguiente forma:

```
tablero = [
    [0,0,3,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,1],
    [0,0,0,0,0,0,0,0,5],
    [0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0],
    [1,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,6,0,0],
    [0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,6,0]
]
```

Esta representación permite acceder fácilmente a filas, columnas y subcuadrículas, facilitando la validación de restricciones y la aplicación de algoritmos de búsqueda.

IV-A2. Variables: Cada celda del tablero representa una variable del problema. Las variables no asignadas corresponden a aquellas posiciones cuyo valor es 0.

IV-A3. Dominios: El dominio de cada variable es el conjunto de valores posibles que puede tomar, es decir, los números del 1 al 9. Este dominio puede reducirse dinámicamente dependiendo de las restricciones del tablero.

IV-A4. Restricciones: Las restricciones del problema están implementadas en el módulo `validacion.py`, específicamente en la función `es_valido`. Estas restricciones son:

- Restricción de fila: un número no puede repetirse en la misma fila.
- Restricción de columna: un número no puede repetirse en la misma columna.
- Restricción de subcuadrícula (3×3): un número no puede repetirse dentro de la subcuadrícula correspondiente.

Estas condiciones se verifican mediante la función:

```
es_valido(tablero, fila, col, num)
```

La cual retorna `True` si el número puede ser asignado sin violar las restricciones, o `False` en caso contrario.

IV-A5. Identificación de variables no asignadas: Para la exploración del espacio de búsqueda, se utiliza la función `encontrar_vacio`, la cual recorre el tablero y retorna la posición de la primera celda sin asignar (valor 0). Esta función es fundamental en los algoritmos de backtracking y forward checking, ya que permite seleccionar la siguiente variable a evaluar.

IV-B. Estructura del Proyecto

El proyecto se organiza de manera modular, separando las diferentes responsabilidades en carpetas y archivos específicos, lo que facilita la comprensión, mantenimiento y reutilización del código.

La estructura general del proyecto es la siguiente:

```
solvers/
    __init__.py
    backtracking.py
```

```
    forward_checking.py
    fuerza_bruta.py

utils/
    __init__.py
    tablero.py
    validacion.py

main.py
```

Carpetasolvers/: Contiene las implementaciones de los algoritmos de solución del Sudoku:

- `fuerza_bruta.py`
- `backtracking.py`
- `forward_checking.py`

Carpeta `utils/`: Contiene funciones auxiliares necesarias para el manejo del problema:

- `tablero.py`: Encargado de la lectura, almacenamiento y manipulación del tablero de Sudoku.
- `validacion.py`: Contiene funciones de validación de restricciones, como la verificación de filas, columnas y subcuadrículas, además de la búsqueda de celdas vacías.

Archivo `main.py`: Actúa como punto de entrada del programa. Se encarga de cargar el tablero, seleccionar el algoritmo de resolución y mostrar la solución obtenida.

IV-C. Implementación

La implementación del proyecto se desarrolló en Python, organizando el código en módulos que permiten separar las responsabilidades del sistema, tales como la lectura del tablero, la validación de restricciones y la aplicación de los algoritmos de resolución.

IV-C1. Lectura del Sudoku: La lectura del tablero se realiza mediante el módulo `tablero.py`, el cual permite cargar un Sudoku desde un archivo de texto o utilizar un tablero predefinido en caso de no especificar una ruta.

El archivo de entrada debe contener una representación del tablero en formato de matriz, donde cada fila está definida entre corchetes y los valores están separados por comas. El valor 0 representa las celdas vacías.

La función principal encargada de esta tarea es `leer_tablero_desde_archivo`, la cual realiza las siguientes operaciones:

- Verifica que el archivo exista.
- Lee el contenido del archivo.
- Extrae las filas utilizando expresiones regulares.
- Convierte los valores a enteros.
- Valida que cada fila tenga exactamente 9 elementos.
- Verifica que los valores estén en el rango permitido (0 a 9).

Si el archivo no cumple con el formato esperado, se generan errores que informan al usuario sobre el problema encontrado.

```
def leer_tablero_desde_archivo(ruta: str)
-> list[list[int]]:
    if not os.path.exists(ruta):
        raise FileNotFoundError(...)
    with open(ruta, "r", encoding="utf-8")
    as f:
    contenido = f.read()
    filas_encontradas =

    re.findall(r'\[([0-9,\s]+)\]', contenido)

    tablero = []
    for fila_str in filas_encontradas:
        numeros = [int(n.strip())
        for n in fila_str.split(",")
        if n.strip()]
        if len(numeros) == 9:
            if any(v < 0 or v > 9 for v
            in numeros):
                raise ValueError(...)
            tablero.append(numeros)
    if len(tablero) != 9:
        raise ValueError(...)

    return tablero
```

Adicionalmente, se implementa la función `cargar_tablero`, la cual permite decidir si se utiliza un archivo externo o un tablero por defecto definido en el sistema. Esto facilita la ejecución del programa en diferentes escenarios.

```
def cargar_tablero
(ruta_archivo: str | None = None)
-> list[list[int]]:
    if ruta_archivo:
        tablero = leer_tablero_desde_archivo
        (ruta_archivo)
        return tablero
return copy.deepcopy(TABLERO_DEFAULT)
```

Para la visualización del tablero, se utiliza la función `imprimir_tablero`, la cual muestra el estado del Sudoku con separadores que permiten identificar claramente las subcuadrículas de 3×3.

Finalmente, el módulo `validacion.py` complementa esta etapa mediante la función `encontrar_vacio`, que permite identificar las celdas sin asignar, facilitando la iteración de los algoritmos de resolución.

IV-C2. Validación de restricciones: Esta función tiene como objetivo verificar si un número puede ser asignado a una celda determinada sin violar las restricciones del Sudoku. Recibe como parámetros el tablero, la posición (fila, columna) y el número a evaluar, retornando un valor booleano que indica si la asignación es válida.

```
def es_valido(tablero: list[list[int]],
fila: int, col: int, num: int) -> bool:
    if num in tablero[fila]:
        return False
```

```
    if num in [tablero[r][col]
    for r in range(9)]:
        return False

    box_fila = (fila // 3) * 3
    box_col  = (col  // 3) * 3
    for r in range(box_fila, box_fila + 3):
        for c in range(box_col, box_col + 3):
            if tablero[r][c] == num:
                return False

    return True
```

La validación se basa en tres tipos de restricciones:

- **Restricción de fila:** verifica que el número no esté presente en la fila correspondiente.
- **Restricción de columna:** comprueba que el número no se repita en la columna.
- **Restricción de subcuadrícula:** asegura que el número no exista en la subcuadrícula de tamaño 3×3 a la que pertenece la celda.

Además, se implementa la función `encontrar_vacio`, la cual permite localizar la siguiente celda sin asignar (valor 0) dentro del tablero. Esta función es utilizada por los algoritmos de resolución para determinar la siguiente variable a evaluar.

```
def encontrar_vacio(tablero:
list[list[int]]) -> tuple[int, int] | None:
    for i in range(9):
        for j in range(9):
            if tablero[i][j] == 0:
                return i, j
    return None
```

En conjunto, estas funciones permiten asegurar que cada paso en el proceso de solución respete las restricciones del problema, evitando la generación de configuraciones inválidas y optimizando la búsqueda de soluciones.

IV-C3. Algoritmo de Fuerza Bruta: La implementación de este algoritmo se encuentra en el módulo `fuerza_bruta.py`, donde se utiliza un enfoque recursivo para recorrer el tablero y asignar valores a las celdas vacías.

A diferencia de otros métodos más eficientes, en esta estrategia la validación de restricciones se realiza después de asignar el valor, mediante la función auxiliar `_es_valido_fb`. Esta función verifica que el número asignado no genere conflictos en la fila, columna o subcuadrícula correspondiente, excluyendo la celda actual de la verificación.

```
def _es_valido_fb(tablero, fila, col, num):
for c in range(9):
if c != col and tablero[fila][c] == num:
        return False
for r in range(9):
if r != fila and tablero[r][col] == num:
        return False

    box_fila = (fila // 3) * 3
    box_col = (col // 3) * 3
for r in range(box_fila, box_fila + 3):
    for c in range(box_col, box_col + 3):
        if (r != fila or c != col)
         and tablero[r][c] == num:
                return False
       return True
```

El algoritmo principal, `solve_sudoku_FB`, funciona de la siguiente manera:

- Busca la siguiente celda vacía utilizando la función `encontrar_vacio`.
- Si no hay celdas vacías, el tablero está completo y se ha encontrado una solución.
- Para cada número del 1 al 9:
  - Se asigna el valor a la celda sin validación previa.
  - Se incrementa un contador de nodos para medir el número de intentos.
  - Se valida la asignación mediante `_es_valido_fb`.
  - Si es válida, se continúa recursivamente con el siguiente paso.
  - Si no conduce a una solución, se realiza *backtracking* (se deshace la asignación).

```
def solve_sudoku_FB(tablero, stats):
    celda = encontrar_vacio(tablero)
    if celda is None:
        return True
    fila, col = celda
    for num in range(1, 10):
        stats["nodos"] += 1
        tablero[fila][col] = num
    if _es_valido_fb(tablero,fila,col,num):
         if solve_sudoku_FB(tablero,stats):
                return True
        tablero[fila][col] = 0
    return False
```

Una característica importante de esta implementación es el uso de un contador de nodos (`stats["nodos"]`), el cual permite medir el número de intentos de asignación realizados durante la ejecución.

IV-C4. Algoritmo de Backtracking: El algoritmo de backtracking es una técnica de búsqueda recursiva que permite resolver problemas de satisfacción de restricciones de manera más eficiente que la fuerza bruta. A diferencia de esta última, el backtracking realiza una validación previa antes de continuar la exploración, descartando de forma inmediata las asignaciones que no cumplen con las restricciones del problema.

La implementación de este algoritmo se encuentra en el módulo `backtracking.py`, donde se emplea la función `es_valido` para verificar si un número puede ser asignado a una celda sin generar conflictos en la fila, columna o subcuadrícula.

El algoritmo funciona de la siguiente manera:

- Se busca la siguiente celda vacía utilizando la función `encontrar_vacio`.
- Si no existen celdas vacías, el tablero está completo y se ha encontrado una solución.
- Para cada número del 1 al 9:
  - Se verifica si el número es válido mediante `es_valido`.
  - Si el valor es válido:
    - Se asigna el número a la celda.
    - Se incrementa el contador de nodos (`stats["nodos"]`).
    - Se continúa recursivamente con el siguiente paso.
  - Si la asignación no conduce a una solución, se deshace el cambio (backtrack).

```
def solve_sudoku_BT(tablero, stats):
    celda = encontrar_vacio(tablero)
    if celda is None:
        return True
    fila, col = celda
    for num in range(1, 10):
    if es_valido(tablero, fila, col, num):

    stats["nodos"] += 1
            tablero[fila][col] = num
    if solve_sudoku_BT(tablero, stats):
                return True
            tablero[fila][col] = 0
    return False
```

Una característica importante de esta implementación es que el contador de nodos se incrementa únicamente cuando se realiza una asignación válida, lo que permite medir con mayor precisión los nodos realmente explorados en el árbol de búsqueda.

IV-C5. Backtracking con comprobación hacia delante (Forward Checking): El algoritmo de Forward Checking es una mejora del backtracking que incorpora técnicas de propagación de restricciones para reducir el espacio de búsqueda. Este enfoque no solo valida las asignaciones actuales, sino que también anticipa conflictos futuros eliminando valores inválidos de los dominios de las variables no asignadas.

La implementación se encuentra en el módulo `forward_checking.py`, donde se combina el uso de backtracking con dos estrategias clave: la propagación de restricciones y la heurística MRV (Minimum Remaining Values).

Inicialización de dominios: Se construye un diccionario que asigna a cada celda el conjunto de valores posibles que puede tomar, considerando el estado actual del tablero. Para las celdas ya asignadas, el dominio contiene únicamente su valor.

```
def inicializar_dominios(tablero):
    dominios = {}
    for i in range(9):
        for j in range(9):
        if tablero[i][j] == 0:
                dominios[(i, j)] = {
                num for num in range(1,10)
            if es_valido(tablero,i,j,num)}
        else:
        dominios[(i, j)] = {tablero[i][j]}
    return dominios
```

Propagación de restricciones: Cuando se asigna un valor a una celda, este se elimina del dominio de sus celdas vecinas (misma fila, columna o subcuadrícula). Si algún dominio queda vacío, la rama se descarta inmediatamente.

```
def forward_checking(dominios,fila,col,num):
    modificados = []
for vecino in obtener_vecinos(fila,col):
        if num in dominios[vecino]:
        dominios[vecino].discard(num)
        modificados.append((vecino, num))
            if len(dominios[vecino]) == 0:
                return False, modificados
    return True, modificados
```

Para garantizar la correcta exploración del espacio de búsqueda, se implementa un mecanismo de restauración de dominios mediante la función `deshacer_forward_checking`, la cual revierte los cambios realizados cuando una rama no conduce a una solución válida.

Heurística MRV: El algoritmo utiliza la heurística MRV para seleccionar la siguiente variable a asignar, eligiendo aquella celda con el menor número de valores disponibles en su dominio. Esto permite reducir el factor de ramificación y mejorar la eficiencia.

```
def seleccionar_variablemrv(dominios,tablero):
    celdas_vacias = [
        (celda, len(dom))
        for celda, dom in dominios.items()
        if tablero[celda[0]][celda[1]]==0
    ]
    if not celdas_vacias:
        return None
return min(celdas_vacias, key=lambdax:x[1])[0]
```

Algoritmo principal: El método `solve_sudoku_FC` integra todas estas técnicas. En cada paso, selecciona una variable usando MRV, prueba los valores de su dominio, propaga las restricciones y continúa recursivamente solo si la asignación es consistente.

```
def solve_sudoku_FC(tablero, stats,
dominios=None):
if dominios is None:
    dominios = inicializar_dominios(tablero)

celda = seleccionar_variable_mrv(dominios,
tablero)
    if celda is None:
```

```
        return True
    fila, col = celda

for num in list(dominios[(fila,col)]):
        stats["nodos"] += 1
if es_valido(tablero,fila,col,num):
tablero[fila][col] = num
dominio_original =
dominios[(fila, col)].copy()
        dominios[(fila, col)] = {num}

exito, modificados =
forward_checking(dominios,fila,col,num)

if exito and
solve_sudoku_FC(tablero,stats,dominios):
                return True
deshacerforwardchecking(dominio,modificados)
dominios[(fila, col)] = dominio_original
        tablero[fila][col] = 0
    return False
```

El uso combinado de propagación de restricciones y heurísticas permite reducir significativamente el número de nodos explorados, haciendo de este algoritmo el más eficiente de las estrategias implementadas en el proyecto.

V. Plan de Pruebas

El plan de pruebas tiene como objetivo verificar el correcto funcionamiento del sistema desarrollado para la resolución del Sudoku, así como evaluar el desempeño de los algoritmos implementados: fuerza bruta, backtracking y backtracking con forward checking. Las pruebas se ejecutaron sobre el tablero por defecto integrado en el sistema y sobre tableros adicionales de distinta dificultad, verificando tanto la correctitud de las soluciones como la eficiencia relativa de cada estrategia.

V-A. Criterios de Aceptación

Para considerar que un caso de prueba es satisfactorio, se establecieron los siguientes criterios:

- La solución producida debe ser válida: ningún valor debe repetirse en su fila, columna ni subcuadrícula de 3×3.
- Los tres algoritmos deben producir la misma solución para el mismo tablero de entrada.
- El sistema debe manejar correctamente entradas inválidas o archivos inexistentes sin interrumpirse de forma inesperada.
- El contador de nodos y el tiempo de ejecución deben registrarse correctamente para cada algoritmo.

V-B. Casos de Prueba

V-B1. Caso 1: Resolución del tablero por defecto: Se ejecuta el programa sin argumentos (`python main.py`), utilizando el tablero integrado en el sistema, que presenta 51 celdas vacías de un total de 81. Se verifica que los tres algoritmos encuentren una solución válida e idéntica, y se registran el tiempo de ejecución y los nodos explorados por cada uno.

Entrada: Tablero DEFAULT.
Resultado esperado: Solución válida producida por los tres algoritmos, con métricas de rendimiento registradas.
Resultado obtenido: Los tres algoritmos produjeron la solución correcta.

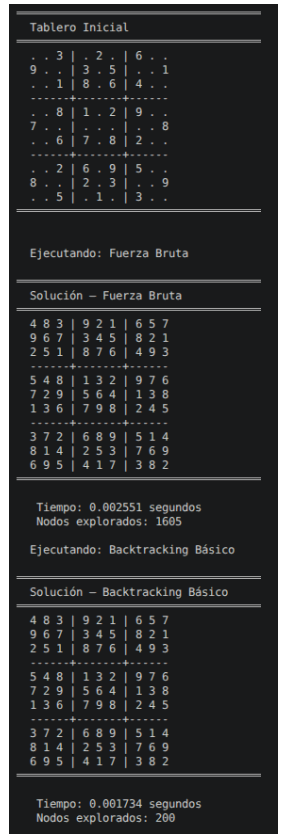

Figura 1. Ejecución completa con el tablero por defecto: tablero inicial, soluciones de los tres algoritmos y resumen comparativo.

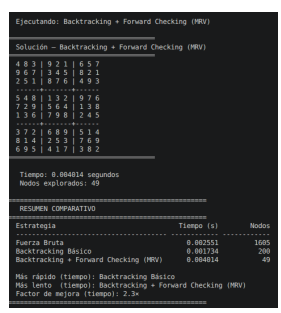

Figura 2. Ejecución completa con el tablero por defecto: tablero inicial, soluciones de los tres algoritmos y resumen comparativo.

V-B2. Caso 2: Lectura de tablero desde archivo externo: Se ejecuta el programa pasando como argumento el archivo `sudoku.txt` incluido en el repositorio (`python main.py sudoku.txt`). El tablero en el archivo es en este caso diferente al DEFAULT, lo que permite verificar que el mó-

dulo de lectura funciona correctamente y produce diferentes resultados.

Entrada: Archivo `sudoku.txt` con el tablero en formato de matriz Python.

Resultado esperado: Tablero cargado correctamente; soluciones diferentes al Caso 1.

Resultado obtenido: El módulo `leer_tablero_desde_archivo` parseó el archivo sin errores y produjo resultados equivalentes.

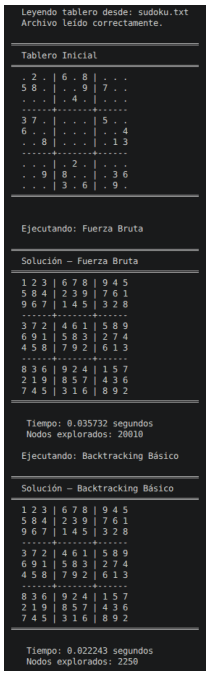

Figura 3. Ejecución con archivo `sudoku.txt`: el sistema confirma la lectura exitosa y resuelve el tablero.

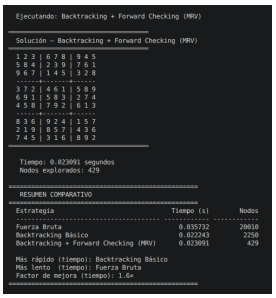

Figura 4. Ejecución con archivo `sudoku.txt`: el sistema confirma la lectura exitosa y resuelve el tablero.

V-B3. Caso 3: Sudoku vacío: Se ejecuta el programa utilizando un tablero completamente vacío, donde todas las celdas contienen el valor 0. Este caso permite evaluar el comportamiento de los algoritmos en el peor escenario posible, debido a que no existe ninguna restricción inicial que reduzca el espacio de búsqueda.

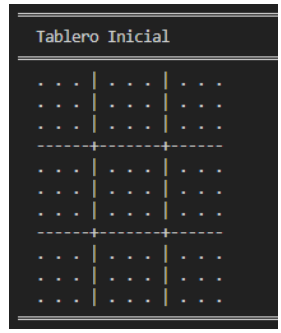

Figura 5. Sudoku Vacío

Se observa que el algoritmo de **Fuerza Bruta** presenta el menor tiempo de ejecución en este caso particular; sin embargo, esto no implica mayor eficiencia, ya que explora un número significativamente alto de nodos (3195), lo que evidencia un amplio recorrido del espacio de búsqueda.

Por otro lado, el algoritmo de **Backtracking Básico** reduce considerablemente el número de nodos explorados (391), gracias a la validación temprana de restricciones.

Finalmente, el algoritmo de **Backtracking con Forward Checking (MRV)** es el más lento en tiempo de ejecución (0.005808 s), debido al costo adicional de mantener y actualizar los dominios. No obstante, es el más eficiente en términos de exploración del espacio de búsqueda, con únicamente 81 nodos.

Estos resultados evidencian que existe un compromiso entre tiempo de ejecución y reducción del espacio de búsqueda, donde algoritmos más avanzados disminuyen la cantidad de nodos explorados a costa de un mayor procesamiento por paso.

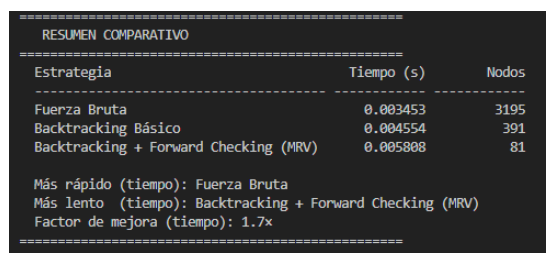

Figura 6. Resumen Comparativo : Se presenta un resumen comparativo del desempeño.

V-B4. Caso 4: Manejo de archivo inexistente: Se ejecuta el programa con una ruta de archivo que no existe (`python main.py archivo_inexistente.txt`), con el objetivo de verificar que el sistema captura la excepción y recurre automáticamente al tablero por defecto sin interrumpirse.

Entrada: Ruta a un archivo inexistente.

Resultado esperado: El sistema emite un `FileNotFoundError`, informa al usuario y continúa con el tablero por defecto.

Resultado obtenido: El error fue capturado correctamente y el programa continuó su ejecución normalmente con el tablero DEFAULT.

Figura 7. Manejo del error por archivo inexistente: el sistema notifica al usuario y utiliza el tablero por defecto.

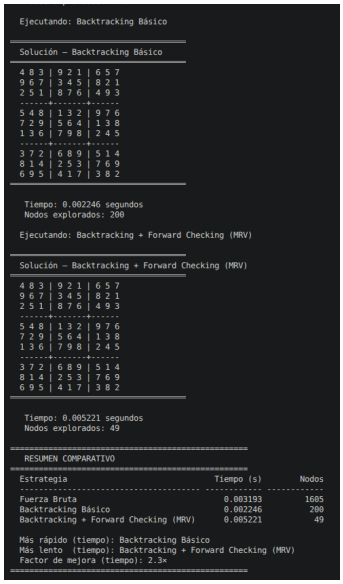

Figura 8. Manejo del error por archivo inexistente: el sistema notifica al usuario y utiliza el tablero por defecto.

V-B5. Caso 5: Escalabilidad con tablero de alta dificultad: Se somete el sistema a un tablero considerado de nivel experto, conocido como *AI Escargot*, que presenta únicamente 22 pistas. Este caso tiene como objetivo evaluar cómo escala cada algoritmo ante un problema de mayor complejidad y evidenciar las diferencias de rendimiento entre las tres estrategias.

Entrada: Tablero *AI Escargot* (22 pistas) cargado desde `dificil.txt`.

Resultado esperado: Los tres algoritmos encuentran la solución; Forward Checking exhibe una ventaja significativa en tiempo y nodos.

Resultado obtenido: Los resultados confirman la superioridad de FC+MRV.

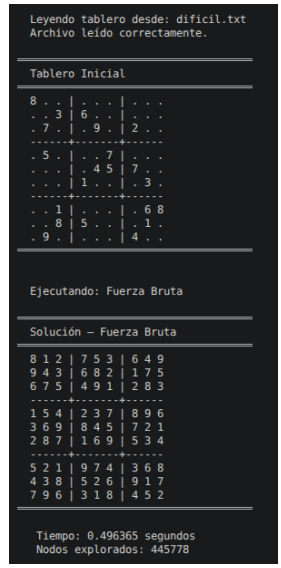

Figura 9. Resumen comparativo para el tablero de alta dificultad: Forward Checking explora 324 veces menos nodos que Fuerza Bruta.

Figura 10. Resumen comparativo para el tablero de alta dificultad: Forward Checking explora 324 veces menos nodos que Fuerza Bruta.

VI. Conclusiones

A partir del desarrollo del proyecto y del análisis de los resultados obtenidos en las pruebas, se presentan las siguientes conclusiones:

El Sudoku constituye un problema de satisfacción de restricciones (CSP) que permite ilustrar con claridad las diferencias en eficiencia entre distintas estrategias de búsqueda. La correcta definición de variables, dominios y restricciones es el punto de partida indispensable para aplicar cualquier técnica algorítmica de resolución.

La **fuerza bruta** demostró ser la estrategia menos eficiente en términos de nodos explorados, ya que asigna valores sin verificar restricciones previamente, explorando ramas inválidas antes de descartarlas. Si bien para tableros simples su tiempo de ejecución puede ser comparable al de métodos más sofisticados, su desempeño se degrada rápidamente al aumentar la dificultad del puzzle: en el tablero *AI Escargot* exploró 445,778 nodos frente a los 1,373 de Forward Checking.

El **backtracking básico** representa una mejora significativa respecto a la fuerza bruta, al validar cada asignación antes de continuar la recursión y descartar inmediatamente las ramas inválidas. Sin embargo, su mayor limitación es que no anticipa conflictos futuros: continúa profundizando en ramas que, aunque no generan conflictos inmediatos, llevan a callejones sin salida más adelante. Esto explica por qué, a pesar de explorar muchos menos nodos que la fuerza bruta, puede presentar tiempos de ejecución más altos en algunos escenarios debido al overhead de Python en la gestión recursiva.

El **backtracking con forward checking y heurística MRV** resultó ser consistentemente el algoritmo más eficiente. La propagación de restricciones hacia los dominios de las variables vecinas permite detectar inconsistencias antes de que ocurran, eliminando ramas inviables sin necesidad de explorarlas. La heurística MRV complementa esta ventaja al seleccionar siempre la variable con menos opciones disponibles, reduciendo el factor de ramificación del árbol de búsqueda. En el tablero estándar exploró únicamente 49 nodos (frente a 200 del backtracking y 1,605 de la fuerza bruta), y en el tablero de alta dificultad su ventaja se amplió a más de 324 veces respecto a la fuerza bruta.

Un aspecto a considerar en la interpretación de resultados es que cada algoritmo implementa una semántica diferente para el contador de nodos: la fuerza bruta y el forward checking cuentan todos los *intentos* de asignación (incluidos los inválidos), mientras que el backtracking solo cuenta las asignaciones válidas. Esto hace que la comparación directa de nodos entre FB y BT no sea completamente homogénea, aunque la tendencia general se mantiene.

En tableros con muy poca información inicial, el espacio de búsqueda crece significativamente, lo que incrementa el tiempo de ejecución de todos los algoritmos. Sin embargo, es precisamente en estos escenarios donde el forward checking demuestra su mayor ventaja: al disponer de dominios más amplios, la propagación de restricciones tiene más margen para eliminar valores inviables de las variables vecinas, reduciendo drásticamente el número de nodos explorados. Los resultados del tablero AI Escargot (22 pistas) ilustran este comportamiento, donde FC exploró 324 veces menos nodos que la fuerza bruta. La única excepción observable es el tiempo de ejecución en tableros muy sencillos, donde el overhead de inicializar y mantener los dominios puede superar el beneficio de la poda, haciendo que el backtracking básico sea marginalmente más rápido en esos casos puntuales.

Finalmente, el proyecto evidenció que la elección del algoritmo de resolución tiene un impacto directo y medible en el rendimiento del sistema, y que técnicas como la propagación de restricciones y las heurísticas de selección de variables son herramientas fundamentales en la Inteligencia Artificial para abordar problemas combinatorios de manera eficiente.

Referencias

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Pearson, 2010.

[2] E. Rich, K. Knight, and S. B. Nair, *Artificial Intelligence*, 3rd ed. McGraw-Hill, 2009.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.